Date:

DAA - Assignment 01
Name: Yash Raj
Roll-no: 21k-0737.

Q#1:

(i) Insertion Sort

18, 7, 25, 14, 3, 21, 10, 30.

In insertion sort the left portion of array is kept sorted. In each iteration, the first element of unsorted portion is selected as the key & inserted into its correct position.

Iteration	Key	Array after insertion.
Initial.	-	18, (7), 25, 14, 3, 21, 10, 30,
1	7	7, 18, (25), 14, 3, 21, 10, 30,
2	25	7, 18, 25, (14), 3, 21, 10, 30,
3	14	7, 14, 18, 25, (3), 21, 10, 30,
4	3	3, 7, 14, 18, 25, (21), 10, 30,
5	21	3, 7, 14, 18, 21, 25, (10), 30,
6	10	3, 7, 10, 14, 18, 21, 25, (30),
7	30	3, 7, 10, 14, 18, 21, 25, 30.

final Sorted Array is:- 3, 7, 10, 14, 18, 21, 25, 30.
Sorted part :- left-side
Unsorted part :- right-side

Ans

PAPER PRODUCT

CamScanner

Roll no: 24k-0737 Date:

(ii) The Key at Each iteration is first Element of unsorted portion.

Iteration	Key-Element	Current-Array
1	7	18, 7, 25, 14, 3, 21, 16, 30.
2	25	7, 18, 25, 14, 3, 21, 16, 30.
3	14	7, 18, 25, 14, 3, 21, 16, 30.
4	3	7, 14, 18, 25, 3, 21, 16, 30.
5	21	3, 7, 14, 18, 25, 21, 16, 30.
6	16	3, 7, 14, 18, 21, 25, 16, 30.
7	30	3, 7, 16, 14, 18, 21, 25, 30.
		3, 7, 16, 14, 18, 21, 25, 30.

(iii) In Worst Case, the array is in reverse sorted order. Each new Key must be compared with & shifted past with Every element in sorted portion. Therefore,

$$\frac{n(n-1)}{2} = \frac{n^2-n}{2}$$

So in Worst Case the TC is $O(n^2)$ Ans.

(iv) Loop-Invariant method:-

Initialization:- Before 1st iteration, only 18 is sorted-portion A Single Element is always sorted hence loop variant is true.

Maintenance:- Comparison.

Termination:- When loop is finished, all elements are in sorted order.

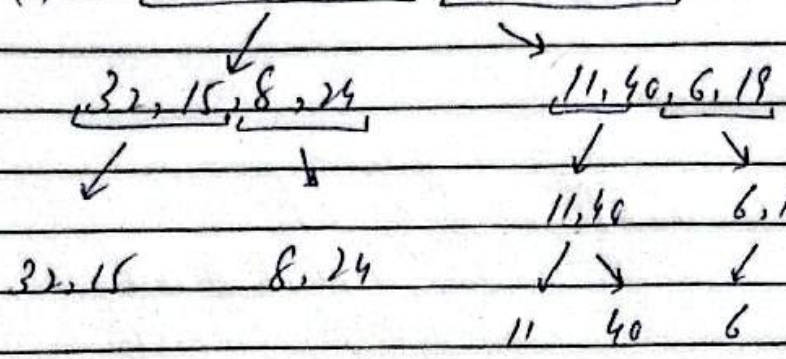
PAPER PRODUCT

CamScanner

Roll no: 24K-0737 Date:

Q#2: Merge sort.

(1) 32, 15, 8, 24, 11, 40, 6, 19.



Recursive sorting & merging:-

[32], [15] → [15, 32].
[8], [24] → [8, 24]
[11], [40] → [11, 40]
[6], [19] → [6, 19]

[15, 32] + [8, 24] = [8, 15, 24, 32]
[11, 40] + [6, 19] = [6, 11, 19, 40]

= [8, 15, 24, 32] + [6, 11, 19, 40].
= [6, 8, 11, 15, 19, 24, 32]

(2) $T(n) = 3T(n/3) + \Theta(n)$.

Sol:- $a = 3, b = 3$
$n^{\log_b a} = n^{\log_3 3} = n$
$f(n) = \Theta(n)$

$T(n) = \Theta(n \log n)$

SP PAPER PRODUCT

CamScanner

Date:

Q#3:- Quick-Sort:-

24, 9, 17, 5, 31, 12, 27, 3.

(i) Pivot:-

pivot = 24.

[9, 17, 5, 12, 3] 24 [31, 27]

Left-Subarray
pivot = 9
[5, 3] 9 [17, 12]

Right-Subarray:-
pivot = 31
[27] 31 [ ].

Left of 9:-
pivot = 5
[3] 5 [ ]

right of 9:-
pivot = 17.
[12] 17 [ ]

[27, 31]

[3, 5, 9, 12, 17].

ii) final sorted Array:- [3, 5, 9, 12, 17, 27, 31] Ans

(ii) Steps Pivot.

1	24
2	9
3	5
4	17
5	31

Ans.

PAPER PRODUCT

CamScanner

Roll-No: 24K-0737 Date:

(iii) Loop-Invariant:

At start of every iteration of partition Element on left < pivot < elements on right of loop

Initialization: Initially, no elements have been processed. So invariant is true.

Maintainance: if element smaller than pivot place on left otherwise right. Thus, invariant remains true

Termination: When loop ends all element before pivot are smaller & after are greater.

Q#4:- Dividing & Conquer:-

10, 15, 7, 20, 12, 18, 9, 14.

(i) $O(n^2)$

1- Initialize Team A & B as Empty.
2- Maintain Sum A & Sum B as skill-rating
3- for Each player:-
if SumA < SumB, put player in TeamA else put in TeamB.
4- Continue until both teams have n players.

Team A: 10, 7, 20, 14 → 51
Team B: 15, 12, 18, 9 → 54

$O(n) + O(n) = O(n^2)$

(ii) $O(n \log n)$

Algo:-

1- Sort all 2n players
2- Take players from highest to lowest
3- Assign highest rated to Team A.
4- & next highest to Team B
5- Continue alternating

TC ∈ $O(n \log n)$

Assignment: $O(n^2)$

PAPER PRODUCT

Date:

Q#5:

Pairing Numbers:-
(2, 4, 6, 8, 10, 12.

1- 2n = 6 n = 3 pairs
(2, 12) → Sum = 14
(4, 10) → Sum = 14
(6, 8) → Sum = 14
Largest pair sum = 14 & it maximum possible.

2- O(nlogn)
(i) Sort 2n number
(ii) pair smallest with largest.
(iii) continue until all paired.
(iv) find largest pair-sum.

T.C
Sorting = O(2nlog(2n)) = O(nlogn).
pairing = O(n)
Therefore = O(nlogn) Ans.

SP PAPER PRODUCT

CamScanner

Roll no: 24k-0737. Date:

Question #6:

Asymptotic proofs:

(a) $n^2 + 5n + 10 \in O(n^2)$.

Soln:

$f(n) = n^2 + 5n + 10$

$g(n) = n^2$

$f(n) \leq c \cdot g(n)$ for all $n \geq 1$

$n^2 + 5n + 10 \leq cn^2$ for some $c \& n_0$.

for $n \geq 1$ $n \leq n^2$

first term $n^2 \leq n^2$ no change.

Second term since $n \leq n^2$ $5n \leq 5n^2$

Third term since $1 \leq n^2$ $10 \leq 10n^2$

$n^2 + 5n + 10 \leq n^2 + 5n^2 + 10n^2$

$16n^2$

$16g(n)$.

$f(n) \leq 16g(n)$ $\forall n \geq 1$

So $c = 16$ $n_0 = 1$

Hence $n^2 + 5n + 10 = O(n^2)$ Ans.

SP PAPER PRODUCT

CamScanner

Date:

(b) $4n^2\log_2 n + 3n^2 \leq O(n^2\log_2 n)$

Sol:- $f(n) = 4n^2\log_2 n + 3n^2$

$g(n) = n^2\log_2 n$

$f(n) \leq g(n)$.

$4n^2\log_2 n + 3n^2 \leq n^2\log_2 n$

for term $\log_2 n$ $n \geq 2$ b/c $\log_2 2 = 1$

$\log_2 n \geq 1$ for every $n \geq 2$.

$\Rightarrow 1 \leq \log_2 n$.

multiply R.S by $3n^2$.

$3n^2 \leq 3n^2\log_2 n$.

$4n^2\log_2 n + 3n^2$.

So, $4n^2\log_2 n + 3n^2 \leq 4n^2\log_2 n + 3n^2\log_2 n$.

$(4+3)n^2\log_2 n$.

$7n^2\log_2 n$.

$f(n) \leq 7 \cdot g(n)$ $\forall n \geq 2$.

So, $C = 7$ $n_0 = 2$

hence

$4n^2\log_2 n + 3n^2 \leq O(n^2\log_2 n)$

SP PAPER PRODUCT

CamScanner

Roll.no: 24K-0737 Date:

Q#2.

1- The main purpose of asymptotic notation is to describe how an algorithm's running time or memory usage growth as input size becomes large. It helps us compare algorithms. Ans.

2- Big O :- Describes an upper bound on growth of an algorithm. meaning running time will not grow faster than a certain range.

Big Ω :- Describes a lower bound meaning algorithm takes at least that much time as input grows.

Big Θ :- Gives a tight bound. It describes the actual growth rate when upper & lower bounds are same. Ans.

3- Constants & lower-order terms are ignored because they have less effect on growth of an algorithm when the input size become very large.

Eg:- $8n^2 + 2n$ $O(n^2)$ because dominating Ans.

PAPER PRODUCT

CamScanner

Date:

4- Ans-

Linear-Search
O(n)

Binary-Search
O(log(n))

Therefore, Binary becomes faster than Linear as input size grows

5- The lecture explained how asymptotic notation is used to analyze & compare algo based on how their performance changes with increasing input-size.

I learned about Big O, Big Ω, Big Θ & concept that constants & lower order terms ignored.

Overall, asymptotic analysis help us understand which algorithm will perform better.

SP PAPER PRODUCT

CamScanner

Roll no: 24K-0737 Date:

Q#8 (iii) $T(n) = 16T(n/4) + n!$

Sol:
$a = 16$
$b = 4$
$f(n) = n!$
$\log_b a = \log_4 16$ $f(n) > n^2$
Case 3: $T(n) = \Theta(f(n))$
$\Theta(n!)$ Ans

(ii) $T(n) = 64T(n/8) - n^2 \log n$

Sol:- (This is not solvable as the $f(n)$ is -ve).
Ans.

(i) $T(n) = \sqrt{2}T(n/2) + \log n$.

Sol: $a = \sqrt{2}$
$b = 2$
$f(n) = \log n$.
$f(n) = \log n$.

$T(n) = \Theta(n^{\log_b a})$
$T(n) = \Theta(n^{\log_2 \sqrt{2}})$
$T(n) = \Theta(\sqrt{n})$ Ans

SP PAPER PRODUCT

CamScanner

Date:

Q#8:- Substitution Method:-

(i) $T(n) = 2T(n/2) + n\log n$

Sol:-

$T(n) = 2T(n/2) + n\log n$ → (i)

$T(n/2) = 2T(n/4) + n/2 \log n/2$ → (ii)

$T(n/4) = 2T(n/8) + n/4 \log n/4$ → (iii)

Substitute (ii) in (i):-

$T(n) = 2[2T(n/4) + n/2 \log(n/2)] + n\log n$

$= 4T(n/4) + n\log n/2 + n\log n$

put $T(n/4)$

pattern:) $T(n) = 2^k T(n/2^k) + n\sum_{i=0} \log(\frac{n}{2})$.

$\frac{n}{2^k} = 1$

$n = 2^k$

$k = \log_2 n$ 2) $T(n) = O(n\log_2 n)$

PAPER PRODUCT

CamScanner

Reg No: 24K-0737 Date:

(ii) $T(n) = 8T(n/2) + n^2$

Sol:

$T(n) = 8T(n/2) + n^2$

$a = 8$

$b = n/2$ $\log_2 8$

$T(n) = 8T(n/2) + n^2$

$T(n/2) = 8T(\frac{n}{4}) + n^2$

$T(n/4) = 8T(\frac{n}{8}) + n^2$

$8^k T(\frac{n}{2^k}) + n^2(1 + 2 \cdots + 4 + 8 \cdots 2^{k-1})$.

$n^2(2^k - 1)$

$n^2(n-1)$

$O(n^3)$

$T(n) = \theta(n^3)$

PAPER PRODUCT

CamScanner

Date:

Q#10.

(a) $f(n) + O(f(n)) = O(f(n))$

So,

$f(n) + O(f(n)) = O((f(n))$.

True.

$f(n) = n$

$O(f(n)) = O(n)$

$n + O(n) = O(n)$

if $n + 3n = 4n$

$4n = O(n)$

So, $n + O(n) = O(n)$

$f(n) + Of(n)) = O(f(n))$ Ans.

(b) $f(n) = O(g(n))$ & $g(n) = O(h(n))$ then $f(n) = O(h(n))$

True:-

This is transitive property of Big-O.

$f(n) \leq c \cdot g(n)$ $\quad$ $g(n) \leq c \cdot h(n)$.

$f(n) \leq c_1 (c_2 h(n))$

$f(n) \leq (c_1 \cdot c_2) h(n)$

c_1, c_2 are constant so,

$f(n) = O(h(n))$

Ex: $n = O(n^2)$ & $n^2 = O(n3)$ then $n = O(n^3)$

PAPER PRODUCT

CamScanner

Roll no: 24K-0737 Date:

(c) $f(n) = \theta(g(n))$ then $f(n) + g(n) = \theta(g(n))$

Sol:

True

$f(n) = \theta(g(n))$ mean $f(n)$ & $g(n)$ have same growth rate therefore adding them doesn't change growth.

$f(n) + g(n) = \theta(g(n))$

e.g:-

$f(n) = n \quad g(n) = n$.

$n + n = 2n$.

Constant ignored $2n = \theta(n)$ &

$f(n) + g(n) = \theta(g(n))$

True

(d) $n^2 = O(n^3)$

Sol: True

n^2 grows no faster than n^3

$n \geq 1 \quad n^2 \leq n^3$

B/c $n^3 = n^2 \cdot n$

& $n \geq 1$ therefore $n^2 = O(n^3)$

e.g $n = 10$

$n^2 = 100 \quad n^3 = 1000$

$100 < 1000$

$n^2 < n^3$

$n = 1$

$n^2 = 1 \quad n^3 = 1$

$n^2 \leq n^3$

True

PAPER PRODUCT

Date:

(e) $n^3 \in O(n^2)$

Sol:-

n^3 grows faster than n^2

for $n > 10$

$n^3 = 1000 \quad n^2 = 100$

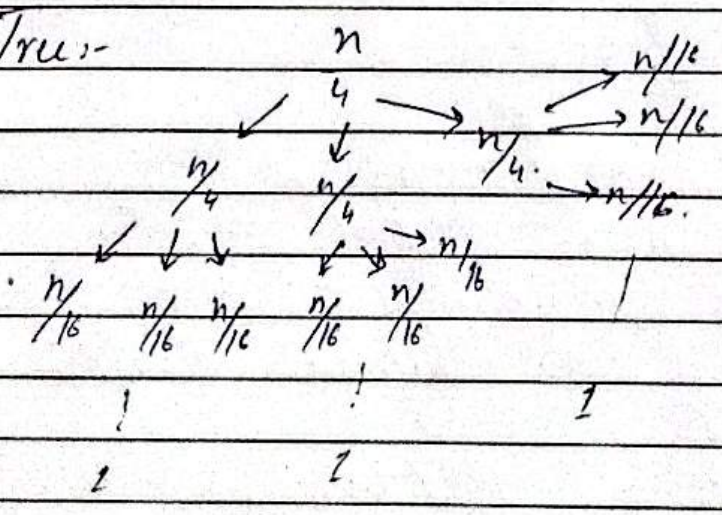
As n get bigger, difference keeps increasing.

$\frac{n^3}{n^2} = n \qquad n^3 \neq O(n^2)$ false

Q#11 :- (a) $T(n) = 3T\left(\frac{n}{4}\right) + n\log n$. $\quad T(1) = O(1)$.

Sol:-

True:-



2) $O(f(n)) = O(n\log n)$

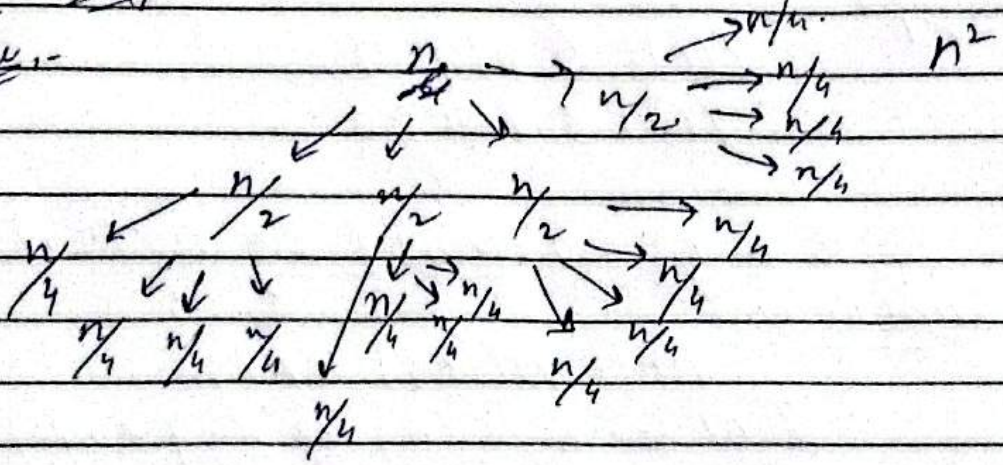
SP PAPER PRODUCT

CamScanner

Roll no: 24k-0737. Date:

(b) $T(n) = 4T(n/2) + n^2$ $T(1) = \Theta(1)$

True:-



level 0 = n^2

level 1 = $\frac{4n^2}{4} = n^2$

level 2 = $\frac{16 n^2}{16} = n^2$

$\frac{n}{2^k} = 1$.

$n = 2^k$.

$\log n = \log 2^k$.

$k = \log_2 n$.

$T(n) = n^2 + n^2 + n^2 \ldots$

$T(n) = n^2 \log n$.

$T(n) = O(n^2 \log n)$.

PAPER PRODUCT

CamScanner

Date:

Question #12 :

$T(n) = 2T(n/2) + n\log n$.

Sol:

$T(n) = \theta(n\log^2 n)$.

$T(n) \le cn\log^2 n$.

if $n = n/2$ 2) $T(n/2) \le c\frac{n}{2}\log^2(n/2)$

Substitute :-

$T(n) = 2T(n/2) + n\log n$.

$T(n) \le 2\left[c\left(\frac{n}{2}\right)\log\left(\frac{n}{2}\right)\right] + n\log n$.

$T(n) \le cn\log^2(n/2) + n\log n$.

$T(n) \le cn(\log n - 1)^2 + n\log n$.

$T(n) \le cn((\log^2 n - 2(n\log n + (n + n\log n)$

$T(n) \le cn\log^2 n + n[(-2c+1)\log n + c]$.

$T(n) \le cn\log^2 n$.

Hence upper bound proved.

Base Case :-

$n = 2$ $T(2) = 2T(1) + 2\log 2$

$T(2) \le 2c\log^2 2$ So both cases proved.

$T(n) = O(n\log_2^2 n)$

proved

PAPER PRODUCT

CamScanner

Roll-no: 24K-0737 Date:

(b) $T(n) = 2T(n/4) + \sqrt{n}$.

Sol:

$a = 2 \quad n^{\log_b a} = n^{\log_4 2} = n^{1/2} = \sqrt{n}$.

$b = 4$

$f(n) = \sqrt{n}$.

$T(n) = \Theta(\sqrt{n} \log n)$.

Induction hypothesis:-

for Upper bound:- $T(n/4) \leq c\sqrt{\frac{n}{4}} \log \frac{n}{4} \quad c > 0$

Inductive step:-

$T(n) = 2T(n/4) + \sqrt{n}$.

$T(n) \leq 2\left[c\sqrt{\frac{n}{4}} \log\left(\frac{n}{4}\right)\right] + \sqrt{n}$.

$\leq c\sqrt{n} \log \frac{n}{4} + \sqrt{n}$.

$T(n) \leq c\sqrt{n}(\log n - 2) + \sqrt{n}$.

$T(n) \leq c\sqrt{n} \log n - 2c\sqrt{n} + \sqrt{n}$

$T(n) \leq c\sqrt{n} \log n + (1 - 2c)\sqrt{n}$.

let $c \geq \frac{1}{2}$

$1 - 2c \leq 0$ therefore:-

$T(n) \leq c\sqrt{n} \log n$.

Base-Case:-

for $n = 1 \quad T(1) = \Theta(1)$

$T(1) \leq c\sqrt{1} \log(1) \Rightarrow T(1) \leq 0$

this creat problem so

$T(n) \leq \sqrt{n} \log n + c\sqrt{n} \Rightarrow T(n) = \Theta(\sqrt{n} \log n)$

PAPER PRODUCT

CamScanner